3.4 Recognition of Tokens

→ In previous section we learned how to express patterns using regular expressions.

→ In this section we are going to study how to take the patterns for all the needed tokens & build a piece of code that examines the input string and finds a prefix that is a lexeme matching one of the patterns.

→ For our discussion we will make use of the following running example:

```
stmt → if expr then stmt
     | if expr then stmt else stmt
     | ε
expr → term relop term
     | term
term → id
     | number
```

Note :-
This is a Pascal language example where then appears after conditions explicitly

→ "relop" is relational operator we use comparison operators in Pascal where "=" is "equals" and "<>" is "not equals".

→ The terminals of grammer, which are if, then

else, relop, id and number are the names of the tokens as far as the lexical analyzer is concerned.

→ The patterns for these tokens are described using regular definitions.

→ The patterns for id and number are.

digit → [0-9]
digits → digit +
number → digits (. digits)? (E [+-]? digits)?
letter → [A-Z a-z]
id → letter (letter | digit)*
if → if
then → then
else → else
relop → < | > | = | <= | >= | <>

→ For this language, the lexical analyzer will recognize the keywords if, then and else, as well as lexemes that matches the patterns for relop, id & number.

→ We assume that keywords are reserved words.

→ We assume that the lexical analyzer will remove the white space characters by recognizing the token "ws" defined by:-

ws → (blank | tab | newline) +

When we recognize any whitespace character we will not return it to the parser, but rather restart the lexical analysis from the character that follows the whitespace. It is the following token that gets returned to the parser.

→ Goal of the lexical analyzer is to find out the token from the lexeme and return it to the parser and make symbol table entry for the token with the attribute name.

So,

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
1) Any WS	–	–
2) if	if	–
3) else	else	–
4) then	then	–
5) Any id	id	pointer to table entry
6) Any number	number	"
7) <	relop	LT
8) <=	relop	LE
9) =	relop	EQ
10) >=	relop	GE
11) <>	relop	NE
12) >	relop	GT

Transition Diagrams

→ As an intermediate step we convert patterns into "transition diagrams".

→ Transition diagrams are collection of "nodes or circles" called states.

→ "State" is nothing but characters which we have seen between BP and FP.

→ We shall assume that all our transition diagrams are deterministic.

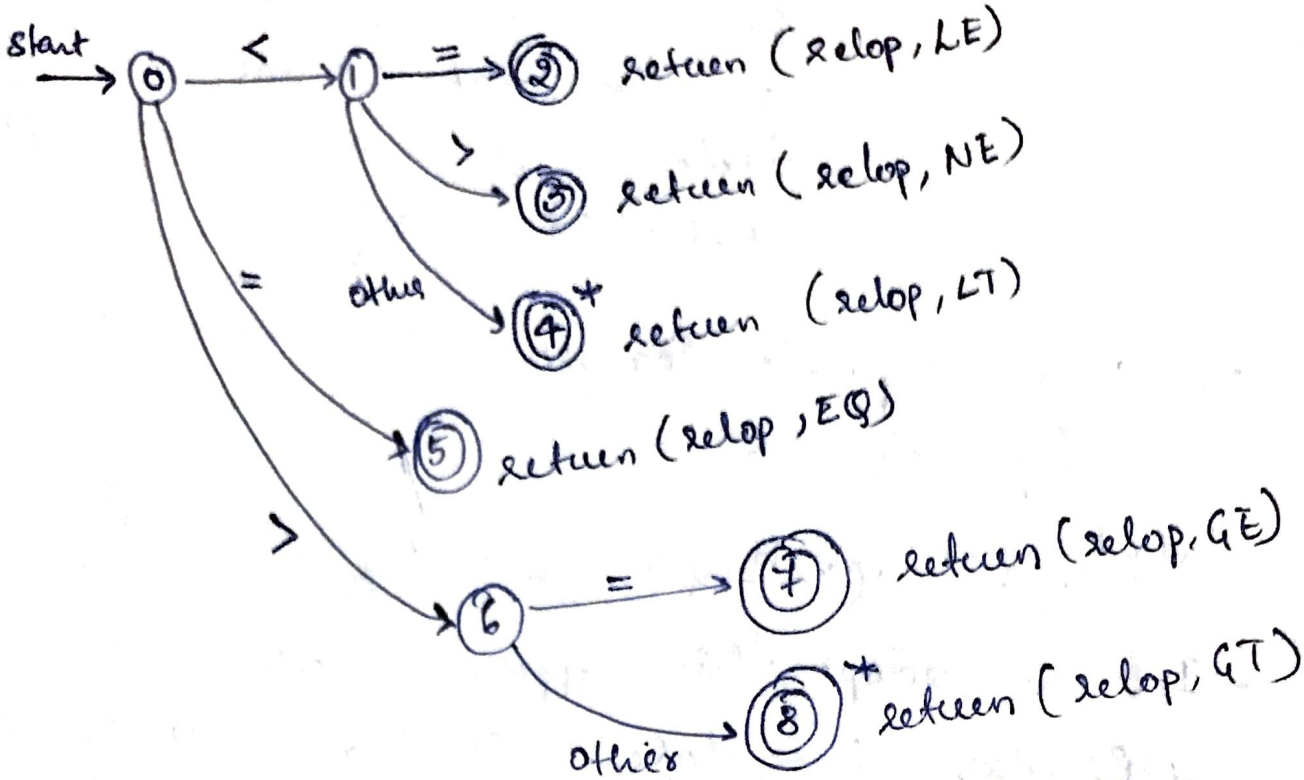
→ Some important conventions about transition diagrams are :-

① Certain states are said to be "accepting" or "final". These states indicates that a lexeme has been found and if any action to be taken (such as returning a token & attribute value to the parser) we shall attach that action to the accepting state.

② If it is necessary to retract the FP one position (i.e. the lexeme does not include the symbol that gets us to the accepting state) then we shall additionally place a * near that accepting state.

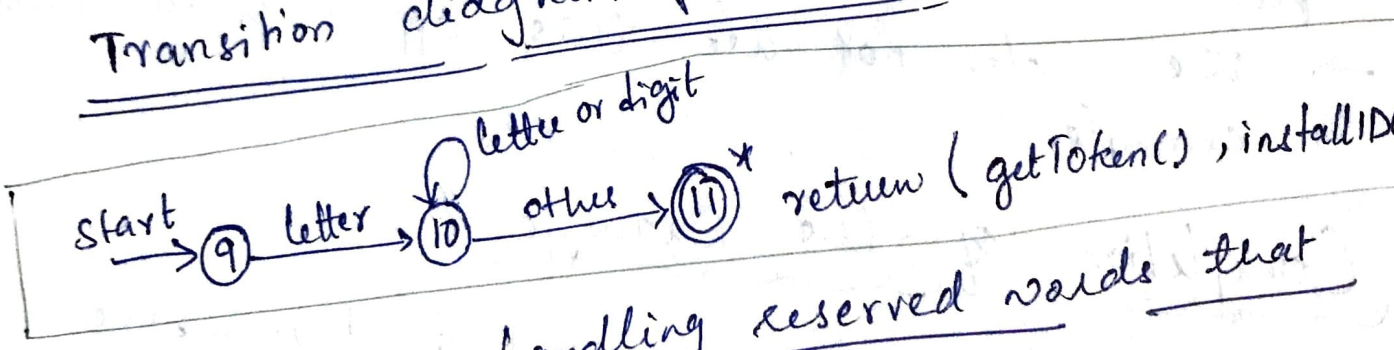
③ One state is "start state". Transition diagram starts in the ~~begins~~ start state before any input symbols have been ~~found~~ read.

Transition diagram for 'relop'



Recognition of Reserved words & Identifiers

Transition diagram for id's & keywords



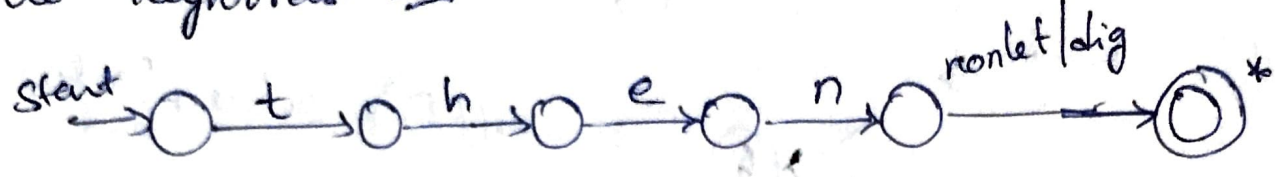
Two ways of handling reserved words that look like identifiers :-

① Install the reserved words in the symbol table initially. A field of the symbol table entry indicat that these strings are never ordinary identi iers, & tells which token they represent.

installID() → installs all the reserved words to the symbol table if it is not present already

getToken() → Examines the symbol table entry for the lexeme found & returns whatever token name the symbol table says this lexeme represents either id or keyword.

2) Create separate transition diagrams for each of the keywords. eg:

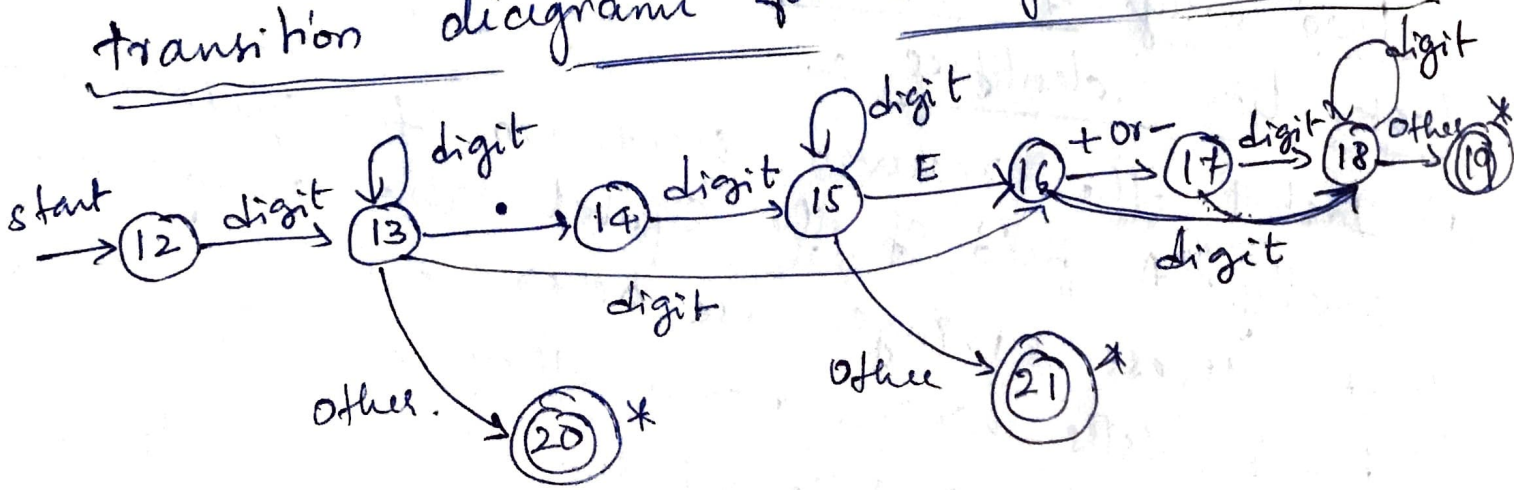


→ If we adopt this approach, then we must prioritize the tokens so that the reserved-word tokens are recognized in preference to id, when the lexeme matches both the patterns.

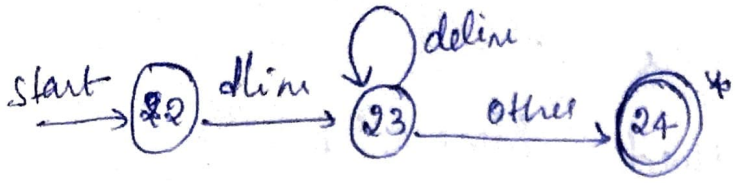
→ We do not use this approach in our example.

Completion of the Example.

transition diagram for unsigned numbers



Transition diagram for whitespace



Architecture of a Transition-Diagram-Based Lexical Analyzer

→ There are several ways that a collection of transition diagrams can be used to build lexical analyzer.

→ Each state is represented by a piece of code.

```
TOKEN getRelop()
{
        TOKEN  retToken = new (RELOP);
        while (1)
        {
            switch (state)
            {
                case 0:   c = nextChar();
                          if (c == '<') state = 1;
                          else if (c == '=') state 5;
                          else if (c == '>') state 6;
                          else fail();  /* lexeme is not
                                           a relop */
                          break;

                case 1:   - - -
                          - - -
                case 8:   retract();  — puts the character back
                          retToken.attribute = GT;
                          return (retToken);
            }
        }
}
```

→ Return value is of TOKEN consisting of a token name and an attribute value.

→ What fail() method does depends on the global error-recovery strategy of the lexical analyzer. It should reset the FP pointer to the BP to allow another transition diagram.

→ If there are no more transition diagrams then fail() could initiate an error correction phase that will try to repair the i/p & find a lexeme.

→ To place the simulation of one transition diagram in perspective the lexical analyzer can be coded in one of the following ways:-

1) We could arrange for the transition diagrams for each token to be tried sequentially. fail() function resets the FP pointer & starts the next transition diagram, each time it is called. It allows us to use transition diagrams for each keywords.

2) We could run the various transition diagrams "in parallel", feeding the next input characters to all of them & allowing each one to make whatever transitions it required.

3) To combine all the transition diagrams into one. Which is used here.